Unit 9: Graph Algorithms (Week 7)

CPTR 242 — Sequential and Parallel Data Structures and Algorithms

Walla Walla University

Part 1: Breadth-First Search

Section 9.1

🤔 You want to find the shortest path between two people in a social network.

"Shortest" means fewest hops, not fastest computation.

How would you systematically explore outward from a source, layer by layer, without missing anyone or revisiting anyone?

9.1 Breadth-First Search

BFS explores a graph level by level from a source vertex, using a queue.

```
Graph:  A - B - D
        |   |
        C - E

BFS from A:  Level 0: A
             Level 1: B, C
             Level 2: D, E
```

Every vertex is visited exactly once. The order guarantees that when vertex v is first reached, the path taken is the **shortest path by hop count**.

9.1 BFS Algorithm

```
void bfs(int src) {
    vector<int> dist(V, -1);
    queue<int> q;
    dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) { dist[v] = dist[u]+1; q.push(v); }
    }
}
```

`dist[v] == -1` serves as the visited check. When BFS finishes, `dist[v]` holds the shortest hop-count distance from `src` to every reachable vertex.

9.1 BFS: Properties and Cost

Property	Value
Time complexity	O(V + E)
Space (queue + visited)	O(V)
Shortest path?	Yes — by hop count
Works on weighted graphs?	No — use Dijkstra
Works on directed graphs?	Yes

> BFS is also used for: checking connectivity, finding connected components, and level-order tree traversal. Any time you need "closest first," reach for BFS.

Part 2: Depth-First Search

Section 9.3

🤔 BFS fans out in all directions simultaneously.

What if you wanted to follow a single path as far as it goes before backtracking — like exploring a maze?

What data structure would naturally support "go deep, then backtrack"?

9.3 Depth-First Search

DFS follows each path to its end before backtracking, using a stack (or recursion).

```
Graph:  A – B – D
        |   |
        C – E

DFS from A (one possible order):  A → B → D → E → C
```

DFS does not guarantee shortest paths.

9.3 DFS Algorithm (Recursive)

```
void dfs(int u, vector<bool>& visited) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v]) dfs(v, visited);
}
```

The call stack implicitly acts as the DFS stack. Each unvisited neighbor is explored immediately and fully before returning to the current vertex.

Part 3: Dijkstra's Shortest Path

Section 9.5

🤔 BFS finds shortest paths by hop count. But roads have lengths.

The path with fewest turns is rarely the fastest route.

How would you always expand the lowest-cost frontier, rather than the earliest-discovered frontier?

9.5 Dijkstra's Algorithm

Dijkstra's finds shortest weighted paths from a source to all vertices, using a min-priority queue.

Core idea: maintain a `dist[]` array initialized to ∞. Repeatedly extract the unvisited vertex with minimum known distance, then **relax** its edges.

Relaxation: if going through u improves the known distance to v, update it.

```
if dist[u] + weight(u,v) < dist[v]:
    dist[v] = dist[u] + weight(u,v)
```

9.5 Dijkstra: Trace

```
Graph: A --4-- B --2-- D
        \      |
         7     1
          \    |
           C--+  (C–B weight 3)

Source: A.  Initial dist: A=0, B=∞, C=∞, D=∞

Extract A(0): relax B→4, C→7
Extract B(4): relax C→min(7, 4+1)=5, D→4+2=6
Extract C(7): no improvement
Extract D(6): done
```

Final: dist = { A:0, B:4, C:5, D:6 }

Critical constraint: Dijkstra requires **non-negative edge weights**.

Part 4: Topological Sort

Section 9.7

🤔 You have a list of tasks where some must be completed before others.

Build before test. Install dependencies before running.

Take CPTR 141 before CPTR 242.

How do you find a valid ordering — and how do you know one exists?

9.7 Topological Sort

A **topological ordering** of a DAG lists vertices so that every edge u → v has u appearing before v.

```
DAG:   CPTR141 → CPTR231 → CPTR242
                      ↓
                  CPTR332

Valid orderings:
  CPTR141, CPTR231, CPTR242, CPTR332  ✅
  CPTR141, CPTR231, CPTR332, CPTR242  ✅
  CPTR231, CPTR141, CPTR242, CPTR332  ❌  (231 before 141)
```

A topological ordering exists **if and only if** the graph is a DAG.

We'll continue Wednesday! 😄 Have a great day!

Part 5: Bellman-Ford

Section 9.9

Bellman–Ford Algorithm

What Bellman–Ford Does

Bellman–Ford finds the **shortest path** from one starting node to every other node.

Unlike Dijkstra's algorithm, it also works when edges can have **negative weights**.

Core Idea

Shortest paths get discovered gradually.

- After 1 pass:
  - shortest paths using **1 edge** are correct
- After 2 passes:
  - shortest paths using **2 edges** are correct
- After 3 passes:
  - shortest paths using **3 edges** are correct

So we repeatedly improve distances until all shortest paths are found.

Key Observation

In a graph with `V` vertices:

- any shortest path can use at most `V - 1` edges

So Bellman–Ford repeats its update process:

```
V - 1 times
```

Initial Setup

Start with:

```
distance[source] = 0
distance[everything else] = ∞
```

Example:

```
A = 0
B = ∞
C = ∞
D = ∞
```

Relaxation

For an edge:

```
u --(w)--> v
```

check whether going through `u` gives a shorter path to `v`.

If:

```
dist[u] + w < dist[v]
```

then update:

```
dist[v] = dist[u] + w
```

This process is called **relaxation**.

Example Graph

```
A --4--> B
A --5--> C
B --(-2)--> C
```

Initial distances:

```
A = 0
B = ∞
C = ∞
```

First Pass

Edge `A → B`

```
0 + 4 < ∞
```

Update:

```
B = 4
```

Continue First Pass

Edge `A → C`

```
0 + 5 < ∞
```

Update:

```
C = 5
```

Continue First Pass

Edge `B → C`

```
4 + (−2) < 5
```

Update:

```
C = 2
```

Final distances after pass:

```
A = 0
B = 4
C = 2
```

Why Multiple Passes?

Distance information spreads outward:

```
Pass 1 → paths using 1 edge
Pass 2 → paths using 2 edges
Pass 3 → paths using 3 edges
```

Eventually all shortest paths become correct.

Example:

```
A → B → C → D

During the first pass:

maybe we discover B

During the second pass:

now we can improve C
```

Negative Cycle Detection

After doing `V - 1` passes:

- check all edges one more time

If any distance can still improve:

```
there is a negative cycle
```

because shortest paths should already be finalized.

Time Complexity

Bellman–Ford processes:

- all edges
- `V - 1` times

Complexity:

```
O(VE)
```

Bellman–Ford in One Sentence

> “Keep relaxing every edge until no shorter paths exist.”

Part 6: Minimum Spanning Tree

Section 9.11

Minimum Spanning Tree (MST)

What Is an MST?

A **minimum spanning tree** is:

- a tree connecting all vertices
- with no cycles
- and the **smallest possible total edge weight**

Example Graph

Edges:

```
A–B = 4
A–C = 7
B–D = 2
C–D = 3
```

What Does “Spanning Tree” Mean?

A spanning tree must:

- connect all vertices
- contain no cycles

With `4` vertices, a tree must contain:

```
4 − 1 = 3 edges
```

So we must remove exactly one edge.

Try Removing Edge A–C

Remove the edge with weight `7`:

```
A ──4── B
        |
        2
        |
C ──3── D
```

Everything is still connected:

```
A → B → D → C
```

Total weight:

```
4 + 2 + 3 = 9
```

Try Removing Edge B–D

```
A ——4—— B
|
7
|
C ——3—— D
```

Total weight:

```
4 + 7 + 3 = 14
```

This is worse.

The Minimum Spanning Tree

The best choice is:

```
A–B = 4
B–D = 2
C–D = 3
```

Diagram:

Total weight: 9

Key Idea

An MST tries to:

```
Keep the graph connected
while using the cheapest possible edges
```

without creating cycles.

Kruskal's Algorithm

Kruskal's: sort all edges by weight; greedily add the cheapest edge that doesn't create a cycle.

```
1. Sort edges by weight ascending
2. For each edge (u, v, w):
     if u and v are in different components:
         add edge to MST
         merge their components
3. Stop when V−1 edges are added
```

Total time: **O(E log E)** — dominated by the sort.

Kruskal’s vs Prim’s Algorithm

What Do They Both Do?

Both algorithms find a:

```
Minimum Spanning Tree (MST)
```

They connect all vertices with:

- no cycles
- minimum total edge weight

Main Difference

Kruskal

> Builds the MST by choosing globally cheapest edges

Main Difference

Prim

> Builds one growing tree outward from a starting vertex

Kruskal’s Algorithm

Idea:

> “Take the cheapest edge available.”

Steps:

1. Sort all edges by weight
2. Add the smallest edge
3. Skip edges that create cycles
4. Continue until all vertices are connected

Kruskal Visual

Start disconnected:

```
A   B   C   D
```

Add cheapest edges:

```
A–B   C–D
```

Merge components:

```
A–B–C–D
```

Prim’s Algorithm

Idea:

> “Grow one tree outward.”

Steps:

1. Start from any vertex
2. Add the cheapest edge leaving the current tree
3. Repeat until all vertices are included

Prim Visual

Start from one vertex:

```
A
```

Grow outward:

```
A–B
```

Continue growing:

```
A–B–D
```

Eventually:

```
A–B–D–C
```

Time Complexity

Algorithm	Complexity
Kruskal	`O(E log E)`
Prim (heap)	`O(E log V)`

9.11 Kruskal's vs. Prim's

Property	Kruskal's	Prim's
Approach	Edge-centric	Vertex-centric
Better for	Sparse graphs	Dense graphs

Both produce a valid MST. Kruskal's is simpler to implement for sparse graphs; Prim's is preferred when $E \approx V^2$.

Part 7: All-Pairs Shortest Path

Section 9.13

Floyd–Warshall Algorithm

What Does Floyd–Warshall Do?

Floyd–Warshall finds:

```
Shortest paths between EVERY pair of vertices
```

Not just from one source.

It computes:

```
distance[u][v]
```

for all vertices `u` and `v`.

Core Idea

Ask this question repeatedly:

> “Would going through vertex `k` make the path shorter?”

For every pair `(i, j)`:

```
Can i → k → j beat i → j ?
```

If yes, update the distance.

Main Formula

For every intermediate vertex `k`:

```
dist[i][j] =
min(dist[i][j], dist[i][k] + dist[k][j])
```

This is the entire algorithm.

What It Means

Current best path:

```
i → j
```

Possible better path:

```
i → k → j
```

If the second one is shorter:

```
update dist[i][j]
```

Triple Nested Loop

Floyd–Warshall is basically:

```
for k in vertices:
    for i in vertices:
        for j in vertices:
            dist[i][j] = min(
                dist[i][j],
                dist[i][k] + dist[k][j]
            )
```

Why It Works

The algorithm gradually allows more intermediate vertices.

- First: paths using no intermediates
- Then: paths allowed to go through vertex 1
- Then: through vertices 1 and 2
- Eventually: through all vertices

At the end, all shortest paths are known.

Time Complexity

There are 3 nested loops:

$$O(V^3)$$

So Floyd–Warshall is slower on large graphs, but very simple.

Negative Edges

Floyd–Warshall works with:

```
negative edge weights
```

as long as there are no negative cycles.

Thanks! Quiz Tomorrow! 😄